Report for Laboratory 6: Realizing a Discrete Dynamic System

Julian Soh
Department of Mechanical Engineering, Saint Martin's University
ME/EE 477—Embedded Real-Time Computing for Mechanical Engineers

July 2, 2026

Abstract. Writing a program capable of approximating the performance of any SISO (Single Input, Single Output), LTI (Linear Time-Invariant) system. Program will use interrupts (see Lab 5) to read the analog input connector of the MyRio that is connected to a function generator, convert the signal from Analog to Digital (ADC), approximate the system's performance and write that to the output of the Digital to Analog (DAC) converter on the MyRio.

1 Introduction

In this laboratory exercise, we developed a general-purpose program to approximate the behavior of any SISO (single-input, single-output), linear time-invariant (LTI) system. The implementation is based on a discrete-time difference equation, allowing the program to emulate a wide range of transfer-function-based system dynamics.

The objectives of this lab were:

1. To use real-time clock interrupts for timing control
2. To implement an arbitrary transfer function generator
3. To incorporate analog-to-digital and digital-to-analog conversion

2 Materials and Methods

2.1 Materials

The materials used in this experiment were:

- Windows 10 computer with myRIO support for C and Eclipse IDE.
- NI myRIO-1900
- Rigol DHO914S Digital Oscilloscope with 25Mhz function generator

2.2 Main Function Implementation

The software architecture uses two concurrent execution paths: a `main()` routine and an interrupt service routine (ISR) running in a dedicated thread. The timer interrupt drives the ISR at a fixed sampling period of 0.5 ms, while the ISR updates the DAC output from the ADC input through a difference-equation-based model.

The `main()` function is intentionally minimal and is responsible for control flow only:

1. Configure and enable the timer interrupt request (Timer IRQ).
2. Remain in a polling loop until the keypad returns a using `getkey()` (implemented previously in Lab 3).
3. Request ISR-thread shutdown by clearing the `irqThreadRdy` run flag, then block until the ISR thread exits cleanly.

2.3 Interrupt Service Routine (ISR) Thread

The ISR thread realizes the discrete dynamic system in real time. Its core logic runs inside a `while` loop that repeatedly checks `irqThreadRdy`; the loop continues while the flag indicates that execution should proceed.

Before entering the loop, the analog I/O subsystem is initialized and analog output connector C channel 1 (AOC1) is forced to 0 V.

For each interrupt cycle, the ISR performs the following sequence:

1. Wait for the next IRQ event, then program the timer for the next interval by writing BTI to `IRQTIMERWRITE` and asserting `TRUE` on `IRQTIMERSETTIME`.
2. Acquire the current sample $x(n)$ from analog input connector C channel 0 (AIC0).
3. Compute the current output $y(n)$ by invoking `cascade()`, which evaluates all biquad stages using the section-by-section difference-equation form.
4. Send $y(n)$ to analog output connector C channel 1 (AOC1).
5. Acknowledge the interrupt so the next cycle can be serviced.

After shutdown, the thread writes the measured response data to `Lab6.mat`.

The ISR allocates and manages the data required by the dynamic model, including:

1. BTI duration in microseconds.
2. Number of biquad sections, n_s.
3. Section coefficients (a_k and b_k) and the stored prior input/output samples.

To keep the cascade manageable, each biquad stage is represented by a structure that stores its coefficients and state history. This makes the full multi-section system easier to configure, evaluate, and maintain.

3 Laboratory Exploration and Evaluation

3.1 Problem L6.1

Write the `main()` program described in Section L6.2, including the timer ISR, but do not implement `cascade()` yet. Instead, complete and debug everything else in the framework for the biquad-cascade implementation. Set up `main()` and the ISR with the required arrays and timing. In the ISR, pass the ADC input directly to the DAC output:

```
VADin = Aio_Read(&AIC0);
Aio_Write(&AOC1, VADin);
```

Use this direct pass-through mode to view input and output on the oscilloscope and verify that interrupt timing is operating correctly.

The full program is available at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab6/main.c.

The input-to-output pass-through behavior was observed on the oscilloscope, as shown in Figure 1.

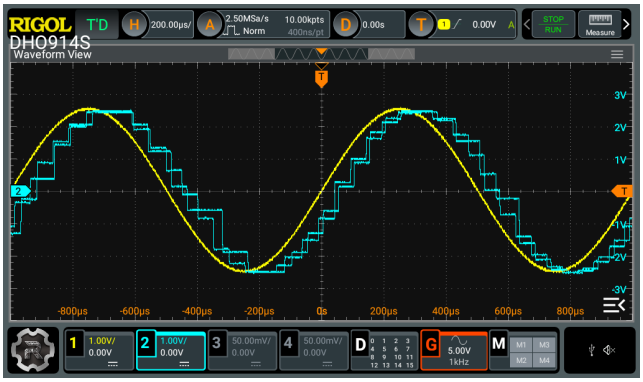

Figure 1: Oscilloscope observation for Problem L6.1 with ADC input passed directly to DAC output.

3.2 Problem L6.2

Implement the `cascade()` function described in Section L6.2. Then extend the `main()` program from Problem L6.1 so that the output signal is computed through `cascade()`.

The full program is available at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab6/main.c.

The oscilloscope response with `cascade()` implemented is shown in Figure 2.

3.3 Problem L6.3

Modify the timer ISR so it stores both the input and the `cascade()` output in 500-point buffers (see Appendix D.1). After the timer loop finishes, export these buffers to `Lab6.mat`.

Instead of using MATLAB, we used Python for post-processing and analysis. Accordingly, the program was modified to capture and save the response data in CSV format.

Figure 2: Oscilloscope observation for Problem L6.2 after implementing `cascade()`.

3.4 Problem L6.4

Using the oscilloscope in DC coupling mode, measure the system step response by applying a low-frequency square-wave input (for example, 8 Hz) with 5 V amplitude from the function generator.

Transfer the response data saved in `Lab6.mat` to your development computer and load it into MATLAB. Plot the measured step response and compare it with the theoretical response of the corresponding continuous-time system. MATLAB's `lsim()` can be used for the theoretical simulation.

Discuss how the measured and theoretical results differ, and explain likely causes of those differences.

The Python post-processing code used for Problem L6.4 is available in the notebook at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab6/Reports/Lab6/data/Step-Response.ipynb.

The oscilloscope step-response measurement was shown in Figure 3.

Figure 3: Oscilloscope cursor measurement for the step response in Problem L6.4.

We used the oscilloscope cursor functions to measure the step response. The cursors were placed at Ay and By at

−2 V and +2 V, corresponding to the 10% and 90% rise-time markers, and Ax and Bx were used to measure the step time. From Figure 3, the oscilloscope showed $\Delta X = \mathbf{10}$ ms.

The Python comparison plot of measured and theoretical step responses is shown in Figure 4.

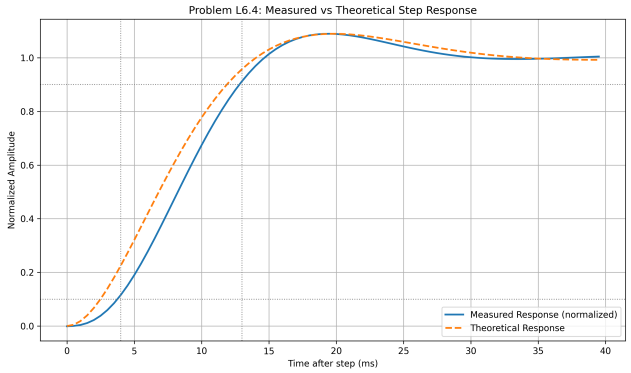

Figure 4: Measured versus theoretical step response for Problem L6.4 (Python post-processing of `lab6.csv`).

The post-processed step-response data in Python gave a measured 10% − 90% rise time of approximately 9 ms, which is consistent with the oscilloscope cursor measurement of about 10 ms. A second-order continuous-time model fitted to the measured waveform produced a response that follows the overall trend well, including a small overshoot and settling behavior. However, the measured waveform is not identical to theory: the measured trace rises slightly more slowly at the beginning and shows small residual ripple/noise near the final value. These differences are expected and are mainly due to non-ideal hardware and implementation effects, including sensor/electronics noise, finite ADC resolution and sampling period, component tolerances, actuator saturation and dead-zone effects, and unmodeled dynamics that are not captured by the ideal continuous-time transfer-function model.

3.5 Problem L6.5

Using the oscilloscope in AC coupling mode, observe the frequency response by varying the frequency of a 5 V sine-wave input.

Record the output amplitude and phase relative to the input at the following frequencies: 5, 10, 20, 40, 60, 100, 140, and 200 Hz.

The frequency-response measurements are shown in Figure 5, Figure 6, Figure 7, Figure 8, Figure 9, Figure 10, Figure 11, and Figure 12.

In each oscilloscope screenshot, the right-side `Result` panel is where the measured values are read: `Vpp(C2)` for output peak-to-peak amplitude and `Phase(r-r)(C1-C2)` for phase.

Table 1: Measured frequency-response values from the oscilloscope in Problem L6.5.

Frequency (Hz)	`Vpp(C2)`	`Phase(r-r)(C1-C2)`
5	4.2330 V	15.347°
10	4.7900 V	32.576°
20	4.9141 V	67.855°
40	3.0537 V	154.59°
60	1.1369 V	-156.44°
100	272.18 mV	-119.27°
140	121.72 mV	-127.62°
200	64.914 mV	-150.28°

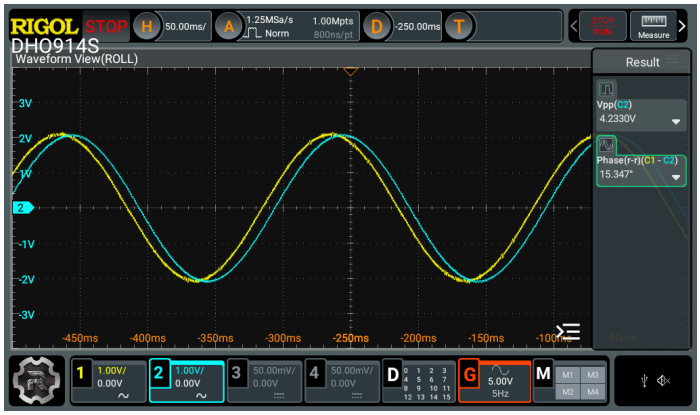

Figure 5: Frequency-response measurement at 5 Hz: `Vpp(C2)` = 4.2330 V, `Phase(r-r)(C1-C2)` = 15.347°.

3.6 Problem L6.6

From the data collected in Problem L6.5, compute the transfer-function magnitude in decibels at each tested frequency.

<findings>

4 Author Contributions

There is only one author for this report and lab.

References

Figure 6: Frequency-response measurement at 10 Hz: `Vpp(C2)` = 4.7900 V, `Phase(r-r)(C1-C2)` = 32.576°.

Figure 7: Frequency-response measurement at 20 Hz: `Vpp(C2)` = 4.9141 V, `Phase(r-r)(C1-C2)` = 67.855°.

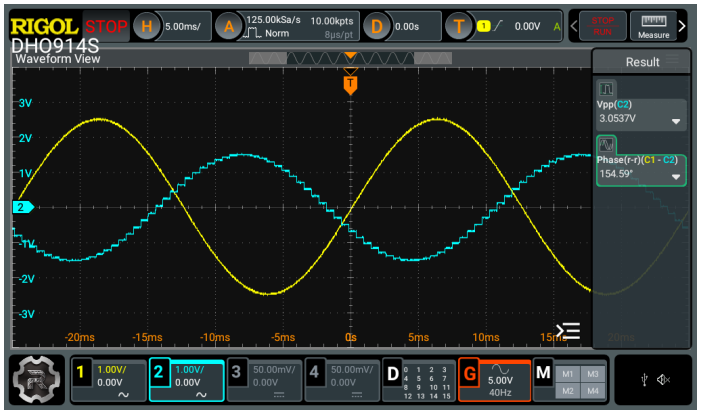

Figure 8: Frequency-response measurement at 40 Hz: `Vpp(C2)` = 3.0537 V, `Phase(r-r)(C1-C2)` = 154.59°.

Figure 9: Frequency-response measurement at 60 Hz: `Vpp(C2)` = 1.1369 V, `Phase(r-r)(C1-C2)` = −156.44°.

Figure 10: Frequency-response measurement at 100 Hz: `Vpp(C2)` = 272.18 mV, `Phase(r-r)(C1-C2)` = −119.27°.

Figure 11: Frequency-response measurement at 140 Hz: `Vpp(C2)` = 121.72 mV, `Phase(r-r)(C1-C2)` = −127.62°.

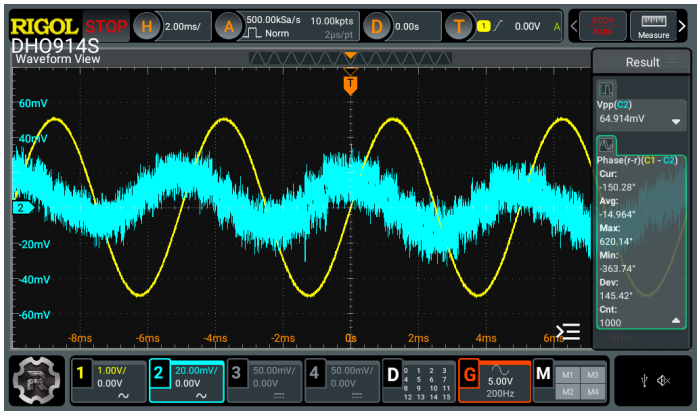

Figure 12: Frequency-response measurement at 200 Hz: `Vpp(C2)` = 64.914 mV, `Phase(r-r)(C1-C2)` current value = −150.28° (phase display fluctuates at this low-amplitude, high-noise condition).